Permutation invariance of the box integral, and the sorted theorem for every multiplicity

Claude, with Daniel Murfet

2026-09-06

Abstract

Lebesgue measure on the box $(0,b]^d$ is invariant under permuting coordinates, so the box integral of unit 54 with exponents $k \circ \sigma, h \circ \sigma$ equals the one with k, h (`boxIntegralFin_perm`). With a bridge from $\mathrm{Fin}\, d$-indexed to $\mathbb{N}$-indexed exponents this turns unit 57's sorted theorem into a statement for any exponent vector that some relabelling sorts (`boxIntegralFin_isEquivalent_of_perm`). The leading asymptotic of the iterated primitives is also extended to $m = 0$, so the sorted theorem now covers a unique minimal exponent (no logarithm) as well. Zero `sorry`/`axiom`.

1 Setting

Mathlib's `measurePreserving_piCongrLeft` states that reindexing a product measure along an equivalence of the index type is measure preserving; for a permutation of $\mathrm{Fin}\, d$ and a constant family of measures both sides are the box measure. The integrand transforms by reindexing the two finite products (`Fintype.prod_equiv`) using `MeasurableEquiv.piCongrLeft_apply_apply`.

2 The things formalised

```
noncomputable def boxIntegralFin (  a b c :  ) {d :  } (k h : Fin d → ) :   :=
   u : Fin d →  , (  i, u i ^ h i) * quadKernel   a (c *   i, u i ^ k i)  (boxMeasure b d)
theorem boxIntegralFin_perm (  a b c :  ) {d :  } (k h : Fin d → )
    (  : Equiv.Perm (Fin d)) :
    boxIntegralFin   a b c (k    ) (h    ) = boxIntegralFin   a b c k h
def toNatFun {d :  } (g : Fin d → ) (v :  ) :   →   :=
  fun i => if hi : i < d then g  i, hi  else v
theorem boxIntegralFin_eq_toNatFun ... :
    boxIntegralFin   a b c k h = boxIntegral   a b c (toNatFun k 1) (toNatFun h 0) d
theorem iterDivPrim_isEquivalent_all (hG : LogBase G A M C    ) (hA : 0 < A) (m :  ) :
    (fun L => iterDivPrim G m L) ~[atTop] fun L => A / (m.factorial :  ) * Real.log L ^ m
theorem boxIntegralFin_isEquivalent_of_perm ... (  : Equiv.Perm (Fin (r + 2 + m))) ... :
    (fun n => boxIntegralFin   a b (Real.sqrt n) k h) ~[atTop]
      fun n => ... * (n ^ (-(p / 2)) * Real.log n ^ m)
```

3 Proof strategy

The permutation lemma is `MeasurePreserving.integral_comp'` applied to the integrand, followed by pointwise reindexing of the two products; the composite equation is assembled with

`Eq.trans` against the (un-beta-reduced) change-of-variables identity rather than `rw`. The $m = 0$ case of the tower asymptotic is `isEquivalent_const_iff_tendsto` plus the `LogBase` tail with `tendsto_rpow_neg_atTop`. The final theorem chains the permutation lemma, the bridge, unit 54's identification with the recursion, and unit 57.

4 Roadblocks and resolutions

- `rw` inside a `congr_left` goal met an un-beta-reduced application; a `beta_reduce` first.
- `rw` with a change-of-variables lemma whose right-hand side is a beta-redex is fragile; `Eq.trans` with the lemma as a term unifies up to beta.

5 What was Mathlib, what was new

Mathlib: `measurePreserving_piCongrLeft`, `MeasurableEquiv.piCongrLeft_apply_apply`, `Fintype.prod_equiv`, `isEquivalent_const_iff_tendsto`, `tendsto_rpow_neg_atTop`. New: the invariance, the bridge, and the relabelled multiplicity theorem.

6 Lessons

The recursion integrates coordinates in a fixed order, so sortedness is an artefact of the proof, not of the statement; making the box form primary and proving permutation invariance once removes it cleanly.

7 Outcome

The multiplicity theorem now applies to any exponent vector that can be relabelled into the sorted form; the remaining step is to show that every exponent vector can (via `Tuple.sort` and the first index attaining the minimum), which would give a hypothesis-free statement with the minimum candidate exponent and its multiplicity read off directly from (k, h).